DS 3 Informatique - 4 décembre : 4h

Ce sujet est composé de trois parties indépendantes.

1 NP-complétude

On rappelle quelques notions de coloration de graphes :

- Une numérotation c d'un graphe $G=(S,A)$ est une fonction de $S \to \mathbb{N}$.
- Une coloration c d'un graphe est une numérotation vérifiant :

$$\forall s,t \in S,\ \ (s,t) \in A \Rightarrow c(s) \neq c(t)$$

- On parle d'une k-coloration s'il s'agit d'une coloration à image dans $[\![0, k-1]\!]$.
- Le nombre chromatique d'un graphe est le plus petit entier k tel que ce graphe admette une k-coloration.

Dans cette partie, on cherche à montrer que le problème général du calcul du nombre chromatique d'un graphe est difficile. Pour ce faire, on étudie plutôt des problèmes de décision liés à la coloration. On définit les problèmes de décision suivants :

- Problème `k-coloration` (pour k fixé)
  - Instance : un graphe G.
  - Question : le graphe G est-il k-colorable ?
- Problème `Coloration`
  - Instance : un graphe G et un entier k.
  - Question : le graphe G est-il k-colorable ?

Par ailleurs, pour A et B deux problèmes de décision, la notation $A \leqslant^p B$ signifie qu'il existe une réduction polynomiale de A à B.

Question 1 : Montrer que pour tout entier k, `k-coloration` $\leqslant^p$ `Coloration`. On justifiera en particulier le caractère polynomial de la réduction proposé.

Correction :
Il suffit de montrer qu'il existe une réduction polynomiale de `k-coloration` à `Coloration`. On propose la réduction suivante :
Si G est une instance de `k-coloration`, on lui associe (G,k) une instance de `Coloration`.

- Il s'agit bien d'une réduction puisque l'instance de `k-coloration` est positive si et seulement si l'instance de `Coloration` associée l'est. En effet, dans le cas où $k \leq |S|$, cela équivaut à dire que G est k-colorable avant et après réduction. Dans le cas où $k > |S|$, le graphe G est k-colorable tout comme les graphe G_1 est 1-colorable.
- Cette fonction est bien polynomiale. En effet, la réduction ne dépend même pas de la taille de l'instance, elle est donc réalisée en temps constant.

Question 2 : Montrer que `2-coloration` $\in$ P. On décrira en français un algorithme polynomial permettant de résoudre le problème.

Correction :
On remarque que si un graphe est 2-colorable, on peut trouver une 2-coloration en réalisant des parcours en profondeur. En effet, un graphe est 2-colorable si et seulement si c'est un graphe biparti.
À chaque appel récursif du parcours en profondeur, la couleur du sommet considéré doit être différente de celle de son prédécesseur. Si l'on rencontre une contradiction en réalisant un appel récursif, le graphe n'est pas 2-colorable. Si l'on a pu parcourir tous les sommets du graphe sans contradiction, le graphe est bien 2-colorable car l'on a construit une 2-coloration.

On représente en C une liste chaînée d'entiers par le type suivant :

```
struct Liste
{
    int val;
    struct Liste* suivant;
};

typedef struct Liste liste;
```

Dès lors, un graphe G sera représenté par un tableau `G` de listes d'adjacence, c'est-à-dire un objet de type `liste**` de telle sorte que si $s \in S$, alors `G[s]` est la liste chaînée des voisins de s. On associera à ce tableau sa taille `n`.

On représentera une numérotation $c : S \to \{0, \ldots, k-1\}$ par un tableau `int*` de taille n contenant des éléments de $\{0, \ldots, k-1\}$, telle que `c[s]` prend la valeur $c(s)$, pour tout sommet $s \in S$.

Question 3 : Écrire une fonction `bool deux_colorable(liste** G, int n)` qui prend en argument un graphe `G` et son nombre de sommets n et renvoie `true` si G est 2-colorable et `false` sinon. Cette fonction devra avoir une complexité linéaire en le nombre de sommets et d'arêtes de G.

Correction :
On introduit la fonction `bool parcours_auxiliaire(liste** G, int n, int* coloration, liste* voisins, int couleur)` qui, étant donnés une graphe G, son nombre de sommets n, un tableau indiquant l'état de coloration des sommets, une liste de voisins et la couleur du sommet étudié, réalise un parcours en profondeur sur les voisins en les 2-colorant et teste si c'est possible.

```
bool parcours_auxiliaire(liste** G, int n, int* coloration, liste* voisins,
    int couleur)
{
    int couleur_oppose = 1-couleur;
    if (voisins == NULL)
    {
        return true;
    }
    bool resultat = true;
    if (coloration[voisins->val] == -1)
    {
        coloration[voisins->val] = couleur_oppose;
        resultat = parcours_auxiliaire(G,n,coloration,G[voisins->val],
            couleur_oppose);
    }
    if (coloration[voisins->val] == couleur)
    {
        return false;
    }
    return (resultat && parcours_auxiliaire(G,n,coloration,voisins->suivant,
        couleur));
}
```

La fonction `deux-colorable` consiste alors simplement à la création d'une coloration initiale vide (représentée par des valeurs de -1 dans chaque case) puis à des appels à `parcours_auxiliaire` sur tous les sommets du graphe.

```
bool deux_colorable(liste** G, int n)
{
    int* coloration = (int*) malloc(n*sizeof(int));
    for (int i = 0; i < n; i++)
    {
        coloration[i]=-1;
    }
    bool resultat = true;
    for (int i = 0; i < n; i++)
```

```
    {
        if (coloration[i] == -1)
        {
            coloration[i] = 0;
            resultat = resultat && parcours_auxiliaire(G,n,coloration,G[i
                ],0);
        }
    }
    free(coloration);
    return resultat;
}
```

Question 4 : Que pouvez vous dire du problème `1-coloration`?

Correction :
Le problème `1-coloration` appartient à `P`. Tester si un graphe est 1-colorable est équivalent à tester si deux sommets de celui-ci sont voisins. En effet, si c'est le cas, le graphe n'est pas 1-colorable puisque ces deux sommets seront de même couleur. Si ce n'est pas le cas, il est bien 1-colorable en fixant tous les sommets à la couleur 0.

Les questions suivantes montrent la `NP`-complétude de `3-coloration`.

Question 5 : Montrer que `Coloration` $\in$ `NP`.

Correction :
Il suffit de montrer qu'il existe un vérificateur en temps polynomial du problème `Coloration`. Étant donné un graphe G, d'un entier k et une numérotation des sommets, on peut vérifier en temps polynomial s'il s'agit bien d'une k-coloration. Il suffit en effet de vérifier que la numérotation est à image dans $[\![0, k-1]\!]$ et que deux sommets voisins n'ont pas la même couleur.

On rappelle les définitions suivantes : pour un ensemble de variables $V = \{v_1, \dots, v_n\}$, un *littéral* est une variable ou sa négation, c'est-à-dire de la forme v_i ou $\overline{v_i}$. Une *clause* est une disjonction de littéraux (séparés par des $\vee$). Une formule est dite en *forme normale conjonctive* si c'est une conjonction de clauses (séparées par des $\wedge$). Elle est dite en 3-FNC si de plus chaque clause contient exactement 3 littéraux. Par exemple, la formule φ_0 suivante est en 3-FNC :

$$\varphi_0 = (a \vee b \vee c) \wedge \overbrace{(\overline{a} \vee c \vee \overline{d})}^{\text{clause}} \wedge (b \vee \underbrace{\overline{c}}_{\text{littéral}} \vee d)$$

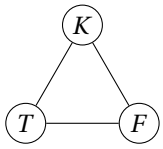
On définit alors le problème de décision :

- Problème `3-SAT`
  - Instance : une formule booléenne φ en 3-FNC.
  - Question : la formule φ est-elle satisfiable?

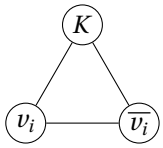
On admet que ce problème est `NP`-complet. La suite de cette partie a pour objectif de montrer qu'il se réduit en temps polynomial au problème `3-coloration`. Pour ce faire, en considérant une formule booléenne φ en 3-FNC, on construit un graphe G_φ qui est 3-colorable si et seulement si φ est satisfiable. L'idée est que deux des couleurs utilisées correspondent à l'affectation *Vrai* ou *Faux* pour chaque littéral, et la troisième couleur est une couleur de contrôle.

Dans le détail, si φ est une formule en 3-FNC sur un ensemble $V = \{v_1, \dots, v_n\}$ de n variables, alors on définit le graphe $G_\varphi = (S, A)$ par :

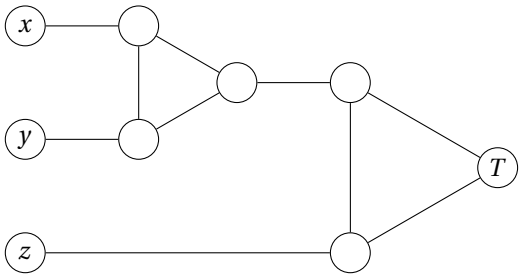
- S contient trois sommets T (*True*), F (*False*) et K (*Control*), un sommet par littéral possible, et 5 sommets supplémentaires par clause ;
- A contient les arêtes suivantes :
  - $\{T, F\}, \{T, K\}, \{F, K\}$



– pour chaque variable v_i, les arêtes $\{v_i, \overline{v_i}\}, \{v_i, K\}, \{\overline{v_i}, K\}$



– pour chaque clause $C = x \vee y \vee z$ apparaissant dans φ (où x, y, z sont des littéraux), on rajoute les 10 arêtes telles que décrites dans le graphe ci-dessous, en utilisant les 5 sommets supplémentaires associés à la clause.



Par exemple, la figure 1 représente le graphe G_{φ_0} obtenu pour la formule booléenne φ_0 sur l'ensemble de variables $\{a, b, c, d\}$.

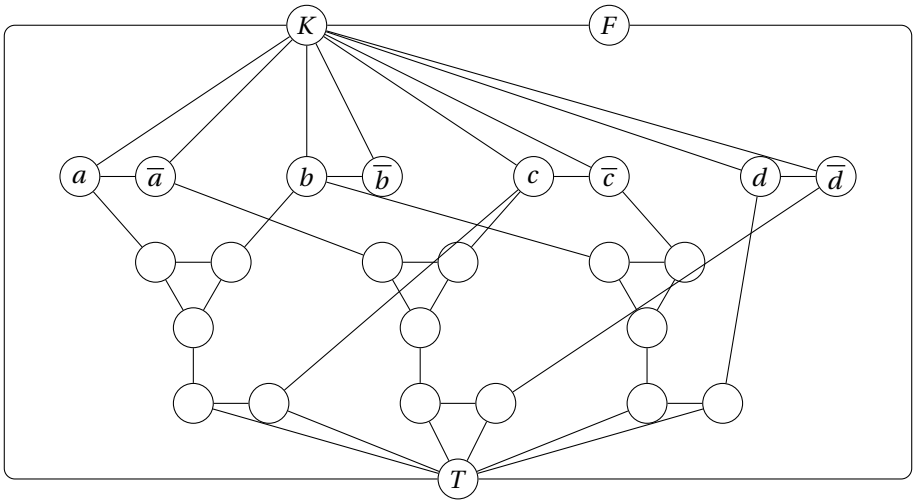


FIGURE 1 – Le graphe G_{φ_0} pour $\varphi_0 = (a \vee b \vee c) \wedge (\overline{a} \vee c \vee \overline{d}) \wedge (b \vee \overline{c} \vee d)$.

Question 6 : Représenter graphiquement le graphe G_{φ_1} pour $\varphi_1 = (\overline{a} \vee b \vee \overline{c}) \wedge (a \vee b \vee \overline{c}) \wedge (a \vee \overline{b} \vee c)$.

Correction :
J'espère avoir des dessins propres à corriger.

Question 7 : Justifier que la construction de G_φ à partir d'une formule φ en 3-FNC est en temps polynomial en fonction de la taille de φ celle-ci étant son nombre de variables et de clauses.

Correction :
Pour chaque variable, on crée deux sommets et 3 arêtes. Le sous-graphe associé à une variable est construit en temps constant. Pour chaque clause, on crée 5 sommets et 10 arêtes. Le graphe associé à une clause est construit en temps constant. On crée en plus de cela 3 sommets et 3 arêtes. La construction du graphe G_φ est donc bien polynomial en la taille de φ puisqu'il s'agit simplement de répéter la création des sous-graphes autant de fois qu'il y a des variables et de clauses.

Question 8 : Montrer que le graphe de la figure 2 est 3-colorable et que pour toute 3-coloration, au moins l'un des sommets x, y ou z a la même couleur que T.

Correction :
Il suffit d'exhiber une 3-coloration du graphe pour montrer qu'il est 3-colorable.
Montrons soigneusement que pour tout 3-coloration, au moins l'un des sommets x, y et z a une couleur identique à celle de T.
Sans perte de généralité, on suppose que K est coloré par 0 et T par 1. On cherche à montrer le résultat par l'absurde donc on suppose que x, y et z sont colorés à 2.
Le voisin commun à T et z est donc coloré à 0. Le sommets situé au dessus est donc coloré à 2. Aucun des sommets du triangle à droite de x et y ne peut être coloré à 2. Or, les trois sommets du triangle doivent être de couleurs différentes. Il est donc nécessaire d'utiliser plus de 3 couleurs.
Donc, x, y ou z admet la même couleur que T.

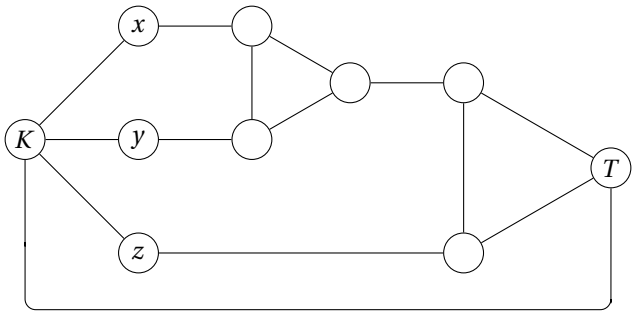


FIGURE 2 – Un gadget de clause.

Question 9 : On suppose que φ est satisfiable. Montrer que G_φ est 3-colorable. On expliquera comment construire une 3-coloration de G_φ à partir d'une valuation de φ.

Correction :
Puisque φ est satisfiable, il existe une valuation v qui satisfait φ.
On colore K à 0, T à 1 et F à 2. On colore les littéraux vrai dans v à 1 et les autres à 2. Montrons qu'on peut compléter cette numérotation pour obtenir une 3-coloration.
On peut alors compléter les gadgets de clause de la façon suivante :
- Les sommets à droite de y et z sont colorés avec 0.
- Les sommets en haut des triangles sont colorés avec la couleur 3-$c(x)$.
- Le dernier sommet est coloré avec la même couleur que x.

On obtient ainsi une k-coloration du graphe.

Question 10 : On suppose que G_φ est 3-colorable. Montrer que φ est satisfiable. On expliquera comment construire une valuation de φ à partir d'une coloration de G_φ.

Correction :
On peut supposer sans perte de généralité que pour la k-coloration de G_φ, K est coloré à 0, T à 1 et F à 2. On définit la valuation v de la façon suivante : x est valué à Vrai par v si et seulement si c colore x à 1.
Il suffit alors de montrer que cette valuation satisfait φ. Cela découle de la propriété prouvée à la question 8.

Question 11 : En déduire que `3-coloration` est NP-difficile, puis qu'il est NP-complet.

Correction :
On a montré que `3-SAT` $\leqslant^p$ `3-coloration`. Puisque `3-SAT` est NP-complet, il est en particulier NP-difficile. Donc, `3-coloration` est NP-difficile.
On a aussi montré `3-coloration` $\leqslant^p$ `Coloration` et `Coloration` $\in$ NP. On en déduit que `3-coloration` $\in$ NP.
`3-coloration` est donc bien NP-complet.

Question 12 : Montrer que pour $h \leq k$ deux entiers fixés, `h-coloration` $\leqslant^p$ `k-coloration`.

Correction :
Étant donnée une instance G de `h-coloration`, on construit une instance G' de `k-coloration` en ajoutant $k-h$ nouveaux sommets adjacents à tous les sommets de G et adjacents entre eux.
Si G' est k-colorable, alors les sommets ajoutés utilisent exactement $k-h$ couleurs distinctes. Aucun autre sommet du graphe n'utilise ces couleurs. G est donc colorable avec h couleurs.
Si G est h-colorable, on peut colorer G' avec k couleurs en assignant aux nouveaux sommets une nouvelle couleur.
Cette réduction est bien polynomiale en la taille de G.

Question 13 : En déduire que pour tout $k \geq 3$, `k-coloration` est `NP-complet`.

Correction :
SOit $k \geq 3$. D'après les questions 11 et 12, on sait que `3-coloration` se réduit polynomialement à `k-coloration` et que `3-coloration` est `NP`-difficile. `k-coloration` est donc `NP`-difficile.
On a aussi montré `k-coloration` $\leqslant^p$ `Coloration` et `Coloration` $\in$ `NP`. On en déduit que `k-coloration` $\in$ `NP`.
`k-coloration` est donc bien `NP`-complet.

2 Automates

L'objectif de cette partie est de proposer quelques éléments autour d'un automate, dit augmenté, construit autour d'un automate fini non déterministe et d'un sous-ensemble d'états.

Un automate fini non déterministe est un quintuplet $\mathscr{A} = (Q, \Sigma, I, \delta, F)$ avec :

- Q un ensemble fini non vide d'états, de cardinal $|Q|$;
- Σ un alphabet ;
- $I \subset Q$ l'ensemble des états initiaux ;
- $\delta : Q \times \Sigma \mapsto P(Q)$ une fonction de transition : si $q \in Q$ et $a \in \Sigma, \delta(q, a)$ désigne l'ensemble des états q' de Q tels qu'il existe une transition étiquetée par a de q vers q' ;
- $F \subset Q$ l'ensemble des états finaux

Soient $\mathscr{A} = (Q, \Sigma, I, F, \delta)$ un automate fini non déterministe et $O \subset Q$ un sous-ensemble d'états de Q. On appelle *automate augmenté* par O la paire $(\mathscr{A}, O)$, que l'on notera par la suite $\mathscr{A}_O$.

La notion de calcul est la même entre $\mathscr{A}$ et $\mathscr{A}_O$: un calcul dans $\mathscr{A}$ est une séquence d'état $q_1 \dots q_n$ de Q telle qu'il existe toujours au moins une transition entre deux états successifs de cette séquence. Le mot construit par concaténation des étiquettes de chacune des transitions est alors reconnu par l'automate.
L'ensemble O introduit la notion de *calcul réussi au seuil s*.

Soit $\mathscr{A}_O$ un automate augmenté. Un calcul passant par les états $q_i \in Q, i \in [\![0, n]\!]$ est dit *réussi au seuil* $s \in \mathbb{N}$ si :

i) $q_0 \in I$;
ii) $q_n \in F$;
iii) pour tout $i \geq 0$ tel que $i + s \leq n, \{q_i \dots q_{i+s}\} \cap O \neq \emptyset$

Soit $\mathscr{A}_O$ un automate augmenté et $s \in \mathbb{N}$. Le *langage reconnu* par $\mathscr{A}_O$ au seuil s, noté $L_s(\mathscr{A}_O)$, est l'ensemble des calculs réussis par $\mathscr{A}_O$ au seuil s.

Correction :
La notion de langage reconnu n'était pas forcément très claire. On pouvait considérer que les mots du langage étaient ceux indiqués par les transitions ou ceux composés des états par lesquels passent l'automate. J'ai corrigé les copies en prenant en compte ce manque de clarté.

On considère, pour les questions 14 à 16, l'automate de la figure 3, où $Q = \{q_0, q_1, q_2\}, \Sigma = \{a, b\}, I = \{q_0\}$ et $F = \{q_2\}$.

Question 14 : Donner sans justifier $L_2(\mathscr{A}_{\{q_0\}})$.

Correction :

$$L_2(\mathscr{A}_{\{q_0\}}) = \{q_0^n q_1 q_2 | n \geq 1\}$$

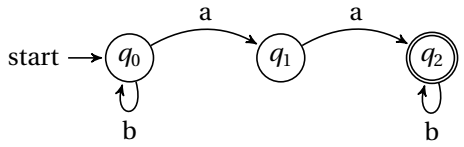


FIGURE 3 – Un exemple d'automate.

Question 15 : Donner, en justifiant votre réponse, $L_2(\mathscr{A}_{\{q_1\}})$.

Correction :
L'ensemble des calculs reconnus (sans notion de seuil) est $\{q_0^i q_1 q_2^j | i > 1, j > 1\}$.
On s'intéresse dans cette question aux éléments de cet ensemble tel que tout facteur de taille 3 utilise la lettre q_1.
On en déduit donc $i \leq 2$ et $j \leq 2$.
On obtient donc :

$$L_2(\mathscr{A}_{\{q_1\}}) = \{q_0^i q_1 q_2^j | (i,j) \in [\![1,2]\!]\} = \{q_0 q_1 q_2, q_0 q_1 q_2^2, q_0^2 q_1 q_2, q_0^2 q_1 q_2^2\}$$

Question 16 : Soit $s \in \mathbb{N}$. Donner, en justifiant votre réponse, le cardinal de $L_s(\mathscr{A}_\emptyset)$ en fonction de s où $\emptyset$ est l'ensemble vide.

Correction :
On raisonne par l'absurde pour montrer que tous les mots de $L_s(\mathscr{A}_\emptyset)$ sont de taille majorée par s. Supposons qu'il existe m un calcul de taille supérieure à $s+1$ appartenant à $L_s(\mathscr{A}_\emptyset)$. D'après la troisième condition d'un calcul réussi au seuil s, il existe une lettre de m qui appartienne à $\emptyset$. C'est contradictoire avec la définition de $\emptyset$.

$$\forall m \in L_s(\mathscr{A}_\emptyset), |m| \leq s$$

Tout mot de taille inférieure à s de $\{q_0^i q_1 q_2^j | i > 1, j > 1\}$ appartient à $L_s(\mathscr{A}_\emptyset)$.

$$|q_0^i q_1 q_2^j| \leq s \Leftrightarrow i + j \leq s - 1$$

$$|L_s(\mathscr{A}_\emptyset)| = |\{q_0^i q_1 q_2^j | i > 1, j > 1, i + j \leq s - 1\}| = \sum_{i=1}^{s-2} \sum_{j=1}^{s-i-1} |\{q_0^i q_1 q_2^j\}| = \frac{(s-2)(s-1)}{2}$$

Question 17 : Donner sans justification, un majorant du nombre de calculs réussis au seuil s ne passant par aucun état de O.

Correction :

$$\sum_{i=1}^{s} |Q|^i = |Q| \frac{1 - |Q|^q}{1 - |Q|}$$

On obtient ce résultat en observant que les calculs réussis au seuil s ne passant par aucun état de O sont tous de taille inférieure à s comme on l'a vu à la question précédente. Il y a $|Q|^i$ calculs distincts de taille i. On en déduit le majorant proposé en sommant sur les tailles possibles.

Question 18 : Soit maintenant un calcul réussi au seuil s qui passe par au moins un état de O. on note $\tilde{q} = q_0 \dots q_l$ la suite des états correspondants et $i_0 \dots i_k \in [\![0, l]\!]$ la suite des indices dans $\tilde{q}$ correspondant aux états de O. Donner, sans justification en fonction de s :

i) une majoration de i_0 ;
ii) un encadrement de i_k ;
iii) une majoration de $i_{j+1} - i_j$ pour $j \in [\![0, k-1]\!]$.

Correction :
On a les inégalités suivantes :
i) $i_0 \leq s$
ii) $l - s \leq i_k \leq l$
iii) $\forall j \in [\![0, k-1]\!], i_{j+1} - i_j \leq s+1$
Chacun de ces résultats est obtenu en utilisant la troisième condition d'un calcul réussi au seuil s.

Question 19 : Soit $\mathscr{A}_0$ un automate augmenté, avec $|Q| = p$, où Q est l'ensemble des états de l'automate. Pour $s \in \mathbb{N}$, construire, en justifiant votre construction, un automate fini à $(s+1)p$ états reconnaissant $L_s(\mathscr{A}_O)$.

Correction :
On cherche à construire un automate qui se souvienne du dernier moment où un état de O a été rencontré.
On construit l'automate A' suivant :
- $Q' = Q \times [\![0, s]\!]$. Les états de A' seront donc des couples (q, n). On cherche en fait à ce que les états caractérisent un état de A et un entier indiquant la dernière rencontre avec un élément de O.
- $\Sigma' = Q$
- $I' = \{(q_0, 0)\}$
- $F' = \{(q, n) | q \in F\}$
- $\delta' : Q' \times \Sigma' \to P(Q')$.

$$\forall (q,n) \in Q', \forall q_i \in Q \text{ tel que } \exists e, q_i \in \delta(q,e), \begin{cases} (q_i, 0) \in \delta'((q,n), q_i) & \text{si } q_i \in O \\ (q_i, n+1) \in \delta'((q,n), q_i) & \text{si } n+1 \leq s \text{ et } q_i \notin O \\ \delta'((q,n), q_i) \text{ n'est pas défini} & \text{sinon.} \end{cases}$$

Lors de la définition de δ', on s'est assuré qu'emprunter une transition $\delta'(q,n) = (q', n')$ de A' conservait les propriétés suivantes :
- $\exists e \in \Sigma, \delta(q,e) = q'$ telle que la transition correspond bien à un calcul de A.
- Si n indique la distance au dernier état de O rencontré depuis (q,n), n' indique la distance au dernier état de O rencontré depuis (q', n').

3 Mots de de Bruijn

Dans cette partie, A désigne un alphabet quelconque. Un *mot de de Bruijn* d'ordre $n \in \mathbb{N}^\star$ est un mot de $A^\star$ qui contient tous les mots de A^n une et une seule fois lorsqu'on le considère circulairement. Par exemple $a\,b\,a\,c\,b\,b\,c\,c\,a$ est un mot de de Bruijn d'ordre 2 sur $\{a, b, c\}$ car il contient une et une seule fois chaque mot de longueur 2 de $\{a, b, c\}$, le mot $a\,a$ étant obtenu en considérant que la lettre suivant le a final est la première lettre du mot, ici a.

Question 20 : Le mot $a\,b\,b\,a$ est-il un mot de de Bruijn sur $\{a, b\}$? Si oui, quel est son ordre ?

Correction :
Le mot $a\,b\,b\,a$ est un mot de de Bruijn d'ordre 2.

Question 21 : Quelle est la longueur d'un mot de de Bruijn en fonction de son ordre n et du nombre k de lettres dans A ?

Correction :
La longueur d'un mot de de Bruijn d'ordre n sur un alphabet de taille k est k^n. En effet, un tel mot possède exactement k^n facteurs de taille n distincts lorsqu'on le considère circulairement. Pour un mot donné, un facteur de taille n est entièrement caractérisé par la position de son premier élément. Il y une bijection entre les facteurs de taille n et les indices des lettres du mot. On en déduit le résultat annoncé.

Les questions suivantes explorent deux méthodes permettant de construire des mots de de Bruijn. La première repose sur la notion de *graphe de de Bruijn*. Le graphe de de Bruijn d'ordre n sur un alphabet A à k lettres est un graphe étiqueté oriente $B(k,n)$ tel que :
- Les sommets de $B(k,n)$ sont étiquetés par les mots de A^n.
- Les arêtes sont données par l'ensemble $\{(a\,m, m\,b, b) | a, b \in A \text{ et } m \in A^\star\}$. Par exemple $(001, 010, 0)$ est une arête dans $B(2,3)$ lorsque l'alphabet est $\{0, 1\}$: elle va du sommet 001 vers le sommet 010 et est étiquetée par 0.

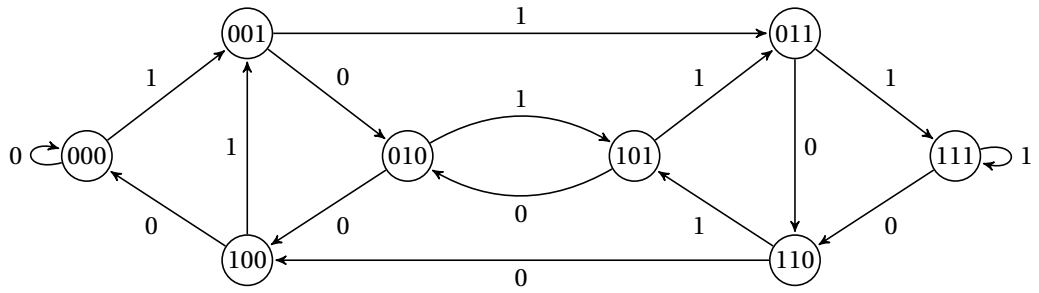


FIGURE 4 – $B(2,3)$

Question 22 : Dessiner le graphe de de Bruijn $B(2,3)$ lorsque l'alphabet est $\{0,1\}$.
Question 23 : Montrer que le degré sortant et le degré entrant de chaque sommet dans $B(k,n)$ est égal à k.

Correction :
Soit s un sommet de $B(k,n)$. Par construction d'un graphe de de Bruijn, il existe $a \in A$ et $m \in A^\star$ tel que l'étiquette de s soit am. La fonction suivante est une bijection :

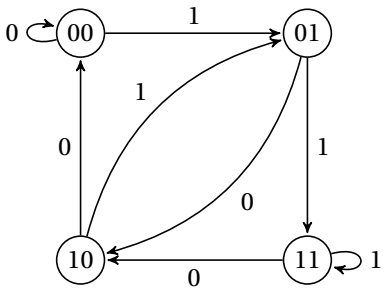
$$\begin{cases} A & \leftarrow & \{\text{arêtes sortants de } am\} \\ b & \mapsto & (am, mb, b) \end{cases}$$

On en déduit qu'il y a autant d'arêtes sortants de s que d'élément de A, à savoir k. Le même raisonnement s'applique sur les arêtes entrantes sur s. Les degrés entrants et sortants sont donc égaux.

Question 24 : En déduire le nombre d'arcs dans le graphe $B(k,n)$.

Correction :
Il y a k arêtes sortantes de chaque sommet. Puisqu'il y a exactement k^n sommets, on en déduit que le graphe $B(k,n)$ possède exactement k^{n+1} arcs.

Un circuit eulérien dans un graphe orienté G est un circuit passant une et une seule fois par chaque arc de G. On admet le résultat suivant : un graphe orienté G admet un circuit eulérien si et seulement si il est connexe et pour tout sommet s de G, les degrés entrant et sortant de s sont égaux.
Question 25 : Construire $B(2,2)$ et déterminer un circuit eulérien dans ce graphe. Constater que le mot obtenu par concaténation des étiquettes des arêtes parcourues lors de ce circuit est un mot de de Bruijn. Quel est son ordre ?



Correction :
Le circuit (01,11,11,10,01,10,00,00,01) est un circuit eulérien. Le mot obtenu en par concaténation des étiquettes des arêtes parcourues est 11010001. C'est un mot de de Brujin d'ordre 3.

Question 26 : Montrer qu'un circuit eulérien dans le graphe $B(k,n)$ produit un mot de de Bruijn d'ordre $n+1$ sur k

symboles.

Question 27 : Montrer que pour tout alphabet A, et tout $n \geq 2$, il existe un mot de de Bruijn sur l'alphabet A d'ordre n.

Correction :
Pour tout alphabet A, et tout $n \geq 2$, on peut construire $B(|A|, n-1)$ le graphe de de Bruijn. Or, $B(|A|, n-1)$ est connexe et pour tout sommet s de G, les degrés entrant et sortants de s sont égaux. $B(|A|, n-1)$ admet donc un cycle eulérien. Ce circuit produit un mot de de Bruijn d'ordre n sur $|A|$ symboles.

L'accès à votre immeuble est protégé par un digicode. Pour entrer, il faut taper une séquence de n chiffres entre 0 et $k-1$. Le digicode fonctionne de la façon suivante : à chaque nouveau chiffre entré après le n-ème, il vérifie si les n derniers chiffres correspondent au code d'entrée. Par exemple, si $n = 4$ et qu'on tape 021201, le digicode teste successivement les codes 0212, 2120 puis 1201.

Question 28 : Combien de codes faudrait-il a priori tester pour pouvoir entrer dans l'immeuble ? Si on les essaie tous successivement, combien de chiffres faudrait-il taper dans le digicode ?

Correction :
Il y a k^n codes possibles. Avec une telle méthode, il faudrait saisir $n \times k^n$ chiffres dans le digicode.

Question 29 : Vu le fonctionnement du digicode, proposer une séquence astucieuse de chiffres qui garantit l'ouverture de la porte tout en diminuant le nombre total de chiffres (qu'on précisera) à entrer dans le digicode. Expliquer brièvement votre raisonnement.

Correction :
On peut utiliser les mots de de Bruijn pour saisir moins de chiffres dans le digicode. On construit m un mot de Bruijn sur $[\![0, k-1]\!]$ d'ordre n. Ce dernier contient toutes les séquences de n chiffres de $[\![0, k-1]\!]$ si on ajoute à la fin ses $k-1$ premières lettres. En effet, ne connaissant pas les derniers chiffres saisis avant le premier de notre mot. Ainsi, il est nécessaire de saisi seulement $k^n + k - 1$ chiffres pour être certain de déverrouiller la porte.